Common errors

Missing authentication

A valid socket ticket is required when opening a socket connection. Learn more about how to obtain a socket ticket and connect to the socket server here.

Connections without a valid ticket will be closed immediately.

Invalid message

```
{
  "res": "v1.error",
  "data": {
    "req": "unknown",
    "message": "Invalid message request",
    "errorCode": "INVALID_MESSAGE",
    "cause": "{\"req\":\"v1.does.not.exist\",\"data\":{}}"
  }
}
```

If you send a `req` that does not exist, you will receive this error. The `req` in the error object is "unknown" because the schema does not match. Check the "cause" for what the failing request looked like.

Invalid message data

```
{
  "res": "v1.error",
  "data": {
    "req": "v1.channels.message",
    "message": "Invalid message data",
    "errorCode": "INVALID_MESSAGE_DATA",
    "cause": "{\"myMessageToTheChannelIsGoingToBeThis\":\"general\"}"
  }
}
```

If the request is valid but the payload data is not, you will receive this message. Check the "cause" to see what you sent and match it up against the schema in the docs.

No player found

```
{
  "res": "v1.error",
  "data": {
    "req": "v1.channels.message",
    "message": "You must identify a player before sending this request",
    "errorCode": "NO_PLAYER_FOUND"
  }
}
```

Most (but not all) requests require the player to be identified before being processed.

Invalid socket token

```
{
  "res": "v1.error",
  "data": {
    "req": "v1.players.identify",
    "message": "Invalid socket token",
    "errorCode": "INVALID_SOCKET_TOKEN"
  }
}
```

When a player is identified using the HTTP API, they receive a `socketToken` to securely connect to the Talo Socket. Socket tokens expire after an hour and are replaced every time the player is identified by the HTTP API.

Invalid session token

```
{
  "res": "v1.error",
  "data": {
    "req": "v1.players.identify",
    "message": "Invalid session token",
    "errorCode": "INVALID_SESSION_TOKEN"
  }
}
```

If you are using Talo Player Authentication for the alias being identified, you will also need to send the `sessionToken` that you receive from the HTTP API after identifying the player. If this is missing, invalid or

expired, you will not be able to identify the player.

Missing access key scopes

```
{
  "res": "v1.error",
  "data": {
    "req": "v1.channels.message",
    "message": "Missing access key scope(s): write:gameChannels",
    "errorCode": "MISSING_ACCESS_KEY_SCOPES"
  }
}
```

Most requests will require your access key to have specific scopes. You will receive this error if they are missing.

Listener error

```
{
  "res": "v1.error",
  "data": {
    "req": "v1.channels.message",
    "message": "An error occurred while processing the message",
    "errorCode": "LISTENER_ERROR",
    "cause": "Player not in channel"
  }
}
```

A request listener can throw an error specific to its own internal logic. These are captured under the generic `LISTENER_ERROR` and the actual error will be highlighted in the "cause".

Rate limit exceeded

```
{
  "res": "v1.error",
  "data": {
    "req": "unknown",
    "message": "Rate limit exceeded",
    "errorCode": "RATE_LIMIT_EXCEEDED"
  }
}
```

Unidentified players are limited to 1 request per second and identified players are limited to 60 requests per second. If this is exceeded, you will receive a rate limit error. After 3 warnings, the connection will be closed and you will need to re-establish a connection to the socket server.